FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Keisha Belle Calibjo
Section:	AN43
Professor:	Crisola G. Tan
Date:	10/10/26

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects

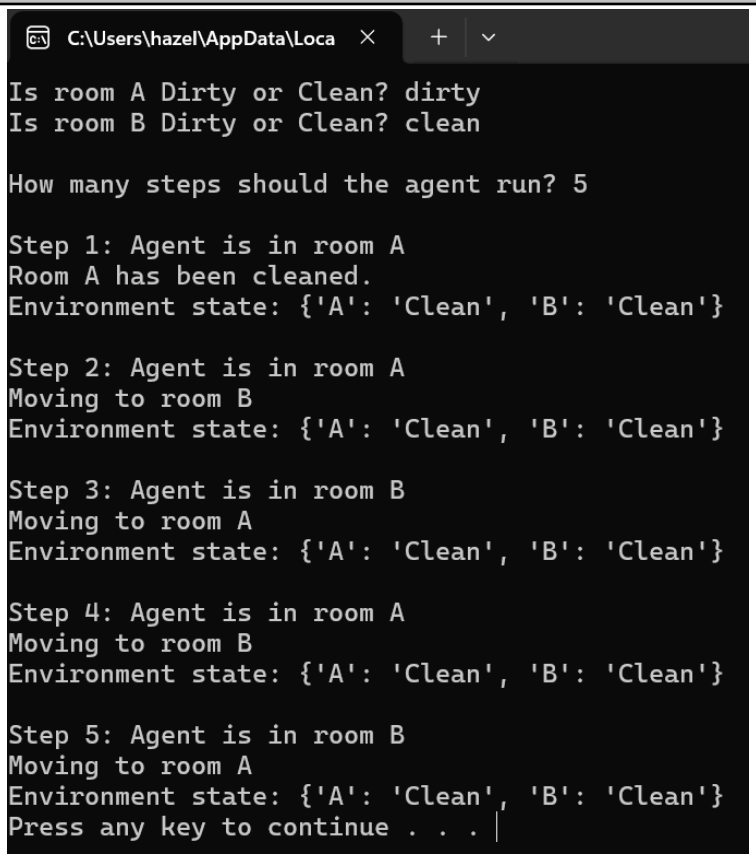
- If-Else statements
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

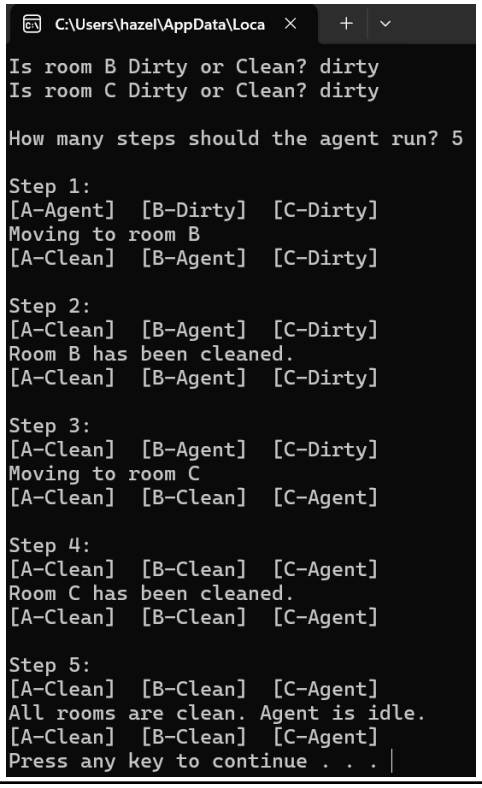
Task 3: Run the Simulation

1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.
3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

agent_simulation.py > VacuumAgent > clean

```python
class VacuumAgent:
    def __init__(self, start_location='A'):
        self.location = start_location

    def perceive(self, environment):
        """Perceives the state of the current room."""
        return environment[self.location]
```

Problems Output Debug Console Terminal Ports ESP-IDF

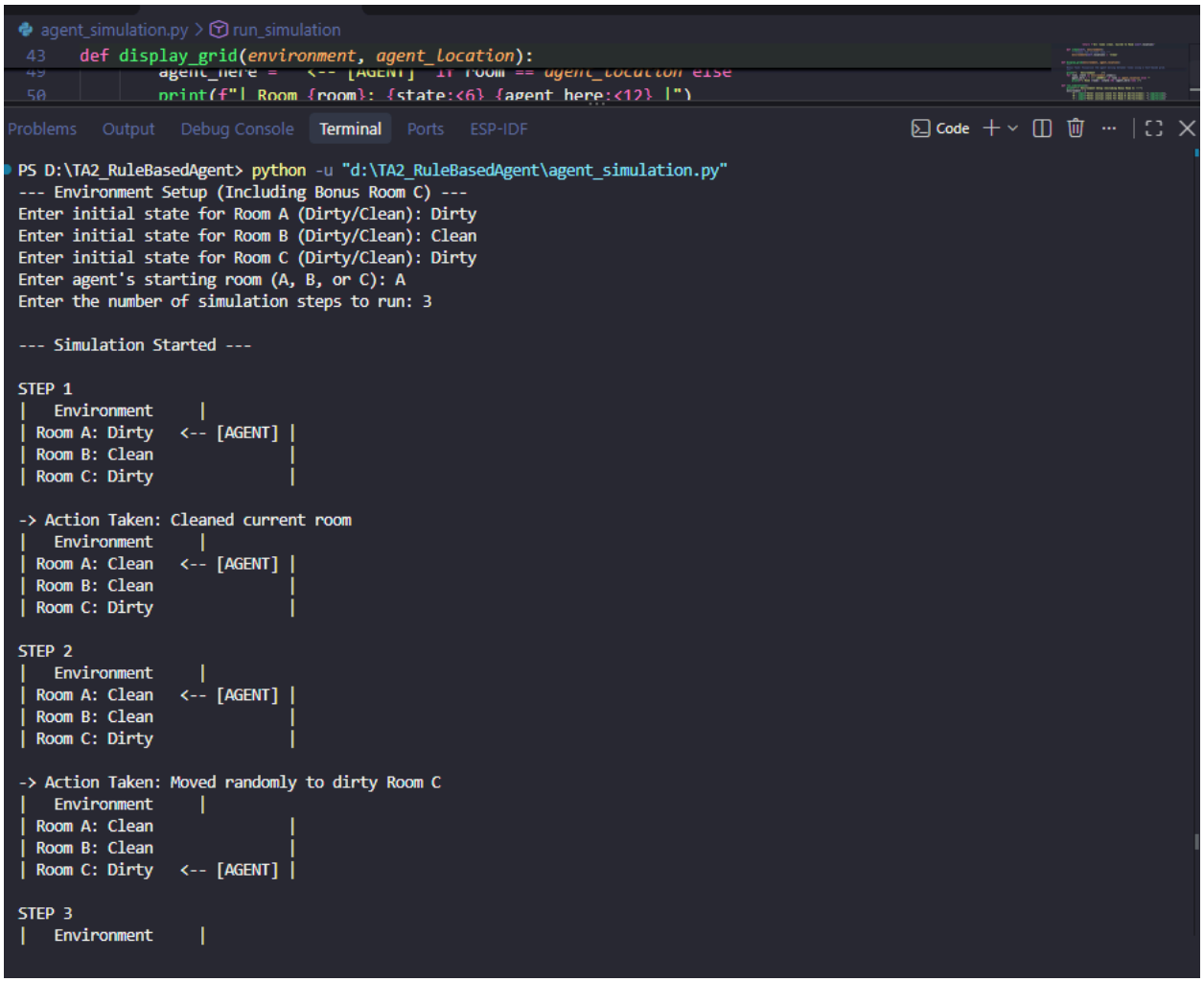
```
PS D:\TA2_RuleBasedAgent> python -u "d:\TA2_RuleBasedAgent\agent_simulation.py"
--- Task 1: Environment Setup ---
Enter initial state for Room A (Dirty/Clean): Clean
Enter initial state for Room B (Dirty/Clean): Dirty
Enter agent's starting room (A or B): A
Enter the number of simulation steps to run: 3

--- Simulation Started ---

Step 1:
Current Environment State: {'A': 'Clean', 'B': 'Dirty'}
Agent's Current Location: Room A
Action Taken: Moved
Updated Environment State: {'A': 'Clean', 'B': 'Dirty'}

Step 2:
Current Environment State: {'A': 'Clean', 'B': 'Dirty'}
Agent's Current Location: Room B
Action Taken: Cleaned
Updated Environment State: {'A': 'Clean', 'B': 'Clean'}

Step 3:
Current Environment State: {'A': 'Clean', 'B': 'Clean'}
Agent's Current Location: Room B
Action Taken: Moved
Updated Environment State: {'A': 'Clean', 'B': 'Clean'}
PS D:\TA2_RuleBasedAgent>
```




```python
import random

class AdvancedVacuumAgent:
    def __init__(self, start_location='A'):
        self.location = start_location

    def perceive(self, environment):
        """Perceives the state of the current room."""
        return environment[self.location]

    def act(self, environment):
        """
        Perceive-and-act method incorporating the bonus logic:
```

```
        - If current room is dirty, clean it.
            - If current room is clean, look for other dirty rooms and
randomly
          jump to one, or just move normally if all are clean.
        """
        status = self.perceive(environment)

        if status == 'Dirty':
            self.clean(environment)
            return 'Cleaned current room'
        else:
            # Bonus Task logic: find all dirty rooms
             dirty_rooms = [room for room, state in environment.items() if
state == 'Dirty']

            if dirty_rooms:
                # Randomly choose which dirty room to visit next
                self.location = random.choice(dirty_rooms)
                return f'Moved randomly to dirty Room {self.location}'
            else:
                # If all rooms are clean, cycle to another room
                rooms = list(environment.keys())
                current_idx = rooms.index(self.location)
                self.location = rooms[(current_idx + 1) % len(rooms)]
                return f'All rooms clean. Cycled to Room {self.location}'

    def clean(self, environment):
        """Cleans the current room."""
        environment[self.location] = 'Clean'


def display_grid(environment, agent_location):
    """
       Bonus Task: Visualize the agent moving between rooms using a
text-based grid.
    """
    print("|   Environment     |")
```

```
    for room, state in environment.items():
        agent_here = " <-- [AGENT]" if room == agent_location else ""
        print(f"| Room {room}: {state:<6} {agent_here:<12} |")

def run_simulation():
    print("--- Environment Setup (Including Bonus Room C) ---")
    environment = {
            'A': input("Enter initial state for Room A (Dirty/Clean):
").capitalize(),
            'B': input("Enter initial state for Room B (Dirty/Clean):
").capitalize(),
            'C': input("Enter initial state for Room C (Dirty/Clean):
").capitalize()
    }

       start_loc = input("Enter agent's starting room (A, B, or C):
").upper()
    if start_loc not in environment:
        start_loc = 'A'

    agent = AdvancedVacuumAgent(start_location=start_loc)

    try:
          steps = int(input("Enter the number of simulation steps to run:
"))
    except ValueError:
        steps = 5

    print("\n--- Simulation Started ---")
    for step in range(1, steps + 1):
        print(f"\nSTEP {step}")

        # Display visual grid before action
        display_grid(environment, agent.location)

        # Agent perceives and acts
        action = agent.act(environment)
```

```
        print(f"\n-> Action Taken: {action}")

        # Display environment state after action
        display_grid(environment, agent.location)

if __name__ == "__main__":
    run_simulation()
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1. Rule 1: If the room is dirty, clean it. Rule 2: If the current room us clean, move to the other room.
2. The agent relies on its perceive method to check the dictionary state of its current room location. If the status reads Dirty, it triggers the cleaning behavior. If the status reads Clean, it triggers the move behavior to switch to the alternate room.
3. If both rooms start clean, the agent will continuously alternate (move) back and forth between Room A and Room B at every time step because it will always perceive a clean room and invoke Rule 2.
4. To handle more rooms (e.g., Rooms A, B, and C), you can expand the environment dictionary and update the move method to cycle sequentially through the rooms example, A → B → C → A or implement a randomized/heuristic strategy to search for and prioritize dirty rooms.